

Session 1 · Deck 1.B3

Why Macroeconomists Need Reinforcement Learning

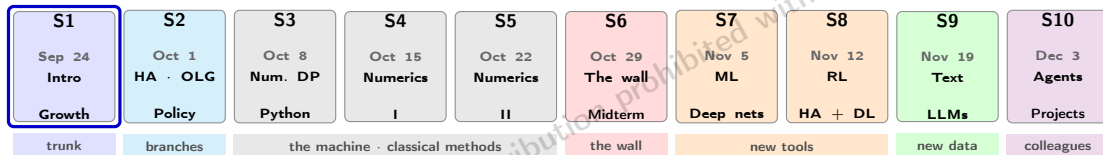
Zhigang Feng

Thursday, September 24, 2026

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are

▼ you are here: 1.B3



Build it on paper → solve it on the machine → push it to the frontier with AI

This deck: dynamic programming quietly assumes you can *write the environment down*. With many agents you cannot — and the two ways dimensionality explodes are what reinforcement learning was built to answer.

Same Equation, Three Worlds

The growth model

2

state variables (k, z)

environment: **written down**

value table: **easy**

solved by VFI (1.B2)

The game of Go

$\approx 10^{170}$

legal positions

environment: **written down**

value table: **impossible**

AlphaGo: a learned move

rule, a value network,

search on a few lines

A heterogeneous-agent economy

∞

a distribution in the state

environment: **not**

written down

value table: **impossible**

the rest of this course

All three are **Bellman equations**. Go separates two things economists usually run together: “the model is known” does *not* mean “the table can be written.” The economy adds a third problem Go does not have — its **environment is made of everyone else’s choices**.

The Assumption Hiding in the Bellman Equation

Every backup in 1.B2 computed the same object:

$$(Tv)(s) = \max_a \{R(s, a) + \beta \mathbb{E}[v(s') \mid s, a]\}$$

$$\mathbb{E}[v(s') \mid s, a] = \sum_{s'} P(s' \mid s, a) v(s')$$

That sum is not free. It asks for P — written down, in closed form, before the first iteration.

In the growth model, it is.

- z' comes from a Markov chain Π — a matrix *you* chose when you discretized the AR(1).
- k' is the action; accumulation then fixes tomorrow's capital exactly.
- Prices are known functions of the state: $r = zf'(k)$, $w = z[f(k) - kf'(k)]$, exactly as in 1.B1.

Three quiet requirements

1. The transition is **writable** — otherwise there is nothing to sum over.
2. The state is **enumerable** — v is a table, swept point by point.
3. No exploration is needed — the max *inspects* every action rather than trying it.

Standing at (k, z) , the only uncertainty is z' , and its distribution is known. All three requirements hold — which is exactly why the theory of 1.B2 applied without comment.

Same Problem, Two Languages

MDP	Growth model
$\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$	
state s	(k, z)
action a	k' (equivalently c)
transition $P(s' \mid s, a)$	k' chosen; $z' \sim \Pi(\cdot \mid z)$
reward $R(s, a)$	$u(c)$
discount γ	β
policy $\pi(s)$	decision rule $k' = g(k, z)$
episode	a simulated path

Economist:

$$V(k, z) = \max_{k'} \{u(c) + \beta \mathbb{E}[V(k', z') \mid z]\}$$

RL researcher:

$$V^*(s) = \max_a \left\{ R(s, a) + \gamma \sum_{s'} P(s' \mid s, a) V^*(s') \right\}$$

The same recursive skeleton, twice. The traditions differ in **what they assume is available**: an economist writes P down; reinforcement learning assumes only that it can *draw samples* from it. That single difference is the whole deck.

Enter Heterogeneity: The Distribution *Is* the Environment

The same equation, one household at a time:

$$V(a, z; Z, \Gamma) = \max_{a' \geq -\underline{a}} \{u(c) + \beta \mathbb{E}[V(a', z'; Z', \Gamma')]\}$$

$$c = (1+r(Z, \Gamma))a + w(Z, \Gamma)z - a', \quad \Gamma' = H(\Gamma, Z, Z').$$

Γ is in the household's problem because **prices are**: r and w come from $K = \int a d\Gamma$ and $L = \int z d\Gamma$. To use its own Euler equation the household must forecast r' — and forecasting r' is forecasting Γ' .

Layer 1: the state explodes

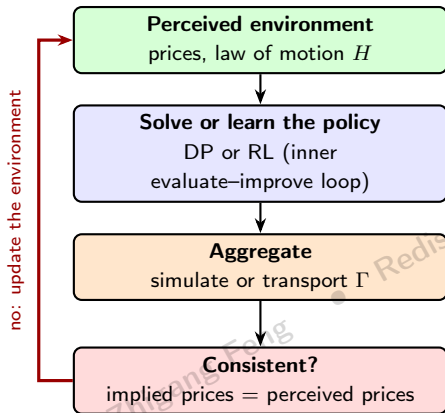
Γ is a measure on assets $\times$ productivity — an infinite-dimensional object. Discretized at 100 asset points and 7 productivity states it is already **700 numbers**, against the growth model's two.

Layer 2: and it is deeper

H has no closed form in general. It is generated by **everyone's policy** g — so the law of motion you must know to solve for g is itself made of g . In macroeconomics, **prices are the environment**, and the environment stops being something you write down.

“No closed form” is an *empirical* statement, not a theorem: in discrete time the kernel can sometimes be written as a matrix and differentiated through (Feng, Han, Sargent & Zhu 2026). S8 does exactly that.

And the Environment Is an Equilibrium Object



- A robot's environment is **fixed**: the room does not change because the robot changed its mind.
- An economy's does. The environment is **made of the policy being learned** — which is the consistency condition of the RCE template from 1.B1, now binding.
- So a heterogeneous-agent model is never “just” a learning problem. It is a learning problem **wrapped in a fixed point**.

Research in Practice

In our taxation project the government is a second learner whose rule reshapes the households' environment — two nested consistency loops.

Dimension 1: Every Extension Adds a State

Extension	What changes in the recursive problem	What it costs
Labour–leisure choice	one intratemporal condition; the state is unchanged	nothing — the rare free lunch
Distortionary taxes, frictions	welfare theorems fail: solve the competitive equilibrium directly	a fixed point
Idiosyncratic risk + borrowing limits	individual state (a, z) ; a stationary distribution λ	one state, one fixed point
Aggregate risk	the distribution enters the state	an <i>infinite-dimensional</i> state
Finite lives	age enters the state; cohort distributions and transitions	one state, plus a transition path
Government without commitment	policy is a rule that must be a best response to itself	another fixed point
Health, job search, entrepreneurship	extra states and discrete choices; non-convexities	a state each, and concavity goes

Every row that makes the economics better makes the computation harder **in the same step**, and the cost is multiplicative: 1.B2 counted it — a hundred points in each of six dimensions is 10^{12} grid points, an 8 TB value table.

Dimension 2: Unfolding Uncertainty Over Time

On a grid, histories merge. Different shock paths land on the same grid point, so rolling a policy forward costs about $|\mathcal{S}| \cdot n_z$ *per period* — it does not grow with the horizon. That is why the Howard steps of 1.B2 are cheap.

That trick needs the table. Once the state is too large to enumerate — once a distribution is in it — you must start from one state and unroll the histories. And histories do not merge; they multiply:

20 periods, 2 shock states	$2^{20} \approx 10^6$
20 periods, 7 quadrature nodes	$7^{20} \approx 8 \times 10^{16}$

There is only one way past an exponential you cannot enumerate: stop enumerating.

The same curse, twice

Direction 1 counts the *states you must cover*; the grid grows like m^d .

Direction 2 counts the *futures you must unroll*; the tree grows like n_z^T .

Different exponents, one cause: exact expectations demand that you visit everything.

And both still require P . The economy of two slides ago fails that condition as well — so all three of the quiet requirements are now gone.

The First Idea: Simulate Instead of Integrate

Full backup — the exact expectation, at every state:

$$Q_{n+1}(s, a) = R(s, a) + \gamma \sum_{s'} P(s' | s, a) V_n(s')$$

Sample backup — simulate N paths and average:

$$\hat{Q}(s, a) = \frac{1}{N} \sum_{i=1}^N G^{(i)}$$

A sample backup needs only a **simulator**: something that turns a state and an action into a next state and a reward. It never asks for P . Its cost grows like $N \times T$ — independent of the number of grid points *and* of the number of histories.

	exact expectation (needs P)	one sample (needs a simulator)
one step	DP / VFI backup	TD(0), Q-learning
whole path	policy evaluation (Howard)	Monte Carlo

In an HA economy. Put ten thousand households in the computer; let all of them save by one candidate rule g ; draw each household's own shock and one aggregate shock. The cross-section is $\hat{\Gamma}_t$; totals follow, and market clearing gives r and w . **Nobody ever wrote H down.**

The Second Idea: Learn the Function, Do Not Tabulate It

Sampling fixes the *expectation*. It does not fix the *table*: you still have to store an answer for every state you might visit, and a 702-dimensional input has no grid.

So stop storing and start fitting. Represent V and g as neural networks V_ψ, g_θ ; train them on the model's own equilibrium conditions — Bellman residuals, Euler residuals, market clearing — evaluated at *sampled* states.

A grid costs m^d numbers. A network's parameter count grows **linearly** in the input dimension.

Say this carefully

This is *not* “neural networks defeat the curse of dimensionality.” A **fixed** basis — Chebyshev polynomials, splines, a grid — cannot beat a rate of $m^{-2/d}$, however cleverly you pick the coefficients. A network also learns *where to put* its basis functions, and on the class of functions where that pays it earns $O(1/m)$ instead (Barron 1993). **The dimension moved into the constant, and into the class; it did not disappear.** 7.A2 makes this precise.

Two substitutions: grids $\rightarrow$ **samples**, tables $\rightarrow$ **functions**. Every method in S7 and S8 is these two moves applied to one model.

What Sampling Costs

Precision. The standard error of a sample mean is $\sigma_G/\sqrt{N}$ — a rate that sees neither d nor the number of histories, but a slow one: **ten times the precision costs a hundred times the sample.**

In the 7×2 table model at $(k, z) = (3.75, 0.9)$: $\sigma_G \approx 0.74$, and the median gap between the best and the second-best action is 0.085. Telling them apart with 95% confidence takes about **400 paths per cell.**

Horizon. An infinite-horizon problem has no terminal state, so a simulated path must be cut. Cut it at T and close with your own estimate, $\sum_{t < T} \gamma^t u_t + \gamma^T \hat{V}(s_T)$.

Both errors move with T : a longer path leans less on that estimate but carries more noise, since σ_G grows with T .

The table model cuts at $T = 300$, uses returns beginning in the first 150 periods, and drops at most $\gamma^{150} \max_s |V^*| \approx 0.003$.

And reusing data is worse

Re-weighting one policy's simulations to evaluate another multiplies T likelihood ratios. One deviation anywhere on the path and the weight is zero:

horizon T	paths carrying any weight
5	65.9%
10	43.3%
20	19.0%
50	1.6%

ε -greedy behaviour policy, $\varepsilon = 0.1$; 500 trajectories per estimate, 200

replications. At $\varepsilon = 0.3$ and $T = 50$, not one usable path survives. Feng &

Miao, *AI for Economists*.

The Other Price: Coverage

At $(k, z) = (3.75, 0.9)$ the five feasible actions are learned from sampled returns, starting from **one path per action**. After 300 rounds, how often is the chosen action the optimal one?

Rule for new samples	optimal	samples/round
pure greedy	0.66	1.0
random re-check (10%)	0.85	1.0
periodic re-check	0.98	1.4
sample every action	1.00	5.0

2,000 replications. Feng & Miao, *AI for Economists*.

Two different problems

Precision: estimates are noisy — this fades as samples accumulate.

Coverage: an action that was unluckily underestimated is never tried again, so it is *never corrected* — this does not fade.

Hence **exploration**. Dynamic programming never faces it: its max inspects every action instead of trying one.

For economists: a central bank cannot explore interest rates for practice, and the Lucas critique bites if it tries. So we train agents inside **calibrated structural models** — which moves the cost of exploration from society to compute.

So When Is It Worth It?

	environment known	environment unknown
low-dim	classical DP: VFI, policy iteration	tabular RL: Q-learning, SARSA
high-dim	“deep DP”: neural nets inside VFI or Euler methods (S7)	deep RL: heterogeneous agents, market design (S8)

Structurally, DP is a limit case of RL: with P known a sample backup becomes a full backup, and with few states a learned function becomes a table.

The honest claim for macro

In macroeconomics the model is usually *known* — we wrote it. Reinforcement learning’s value here is **not** that it is model-free. It is **simulation instead of integration** and **learned functions instead of grids**, in dimensions where the alternatives have no answer at all.

And the guarantees thin out

The contraction argument of 1.B2 covers *tables*. Combine function approximation, bootstrapping and off-policy data and convergence is no longer guaranteed. **Verification stops being a courtesy and becomes the method.**

How This Course Handles the Distribution

Strategy	What it assumes / does	Examples
Stationary equilibrium	No aggregate risk: λ and prices are constant. Inner DP for households, outer loop on the interest rate.	Huggett; Aiyagari
Approximate aggregation	Agents forecast prices from a few moments of Γ ; iterate on the forecasting rule.	Krusell–Smith (1998)
Linearize around the stationary distribution	Small aggregate shocks; perturbation in the distribution itself.	Reiter (2009); sequence-space Jacobian
The distribution as a network input	Histograms or learned moments feed neural networks; simulate or transport Γ exactly.	DEQN; DeepHAM; actor–critic with an analytic kernel

Four answers to one question — **what must agents forecast, and what makes the forecast correct?** The first three shrink the environment until it can be written down. The fourth stops trying.

Session 1 in One Slide — and Next Week

1. The growth model is the trunk: planner, market and recursive equilibrium coincide, and the **RCE template** — state, forecast, consistency — organizes every later model.
2. Dynamic programming is trustworthy and computable, and it assumes you can **write the environment down**.
3. With many agents you cannot: prices depend on a distribution, and the distribution's law of motion is made of the policy you are solving for.
4. Dimensionality then bites twice — across the **states** you must cover, and across the **futures** you must unroll.
5. Reinforcement learning answers both with two substitutions — **sample** the expectation, **learn** the function — and hands you a new job: **verification**.

Next week (S2): why can't everyone insure?

Bewley–Huggett–Aiyagari, Krusell–Smith, overlapping generations, and optimal policy with and without commitment.

Exit ticket before you leave: one thing that clicked today, one thing that did not.

Before then: Ljungqvist–Sargent ch. 3–4 or

Stokey–Lucas–Prescott ch. 4; Sutton–Barto ch. 3–4 (optional); install Python with the setup lab.